

Home ▶ STM32 Tutorials ▶ FreeRTOS Tutorials ▶ **Queue: Inter-Task Communication**

Updated On: May 5, 2026

STM32 FreeRTOS Queue Tutorial: Send Data Between Tasks Using CMSIS-OS V2

In the [previous tutorial](#), we looked at how to create multiple FreeRTOS tasks with different priorities on STM32. Now that we have tasks running in parallel, the next challenge is getting them to talk to each other.

We can do this using global variables. It seems simple — just declare a variable, let one task write to it, and let another task read it. But this approach breaks down fast in a multi-task environment, and it can cause subtle bugs that are very hard to track down.

In this tutorial, we will use **FreeRTOS queues** — the proper way to pass data between tasks. We will create two sender tasks and one receiver task. Both senders will write data into a shared structure and push it into a queue. The receiver will read from that queue and print the output through UART using `printf`.

We are working on the STM32L496 Nucleo board, but the same steps apply to any STM32 board running FreeRTOS with CMSIS-OS v2.

This is the **Part 3** of the [STM32 CMSIS-RTOS FreeRTOS series](#). You can go through the other parts of this series, here are the links:

- **Part 1** — Configure CMSIS RTOS in STM32
- **Part 2** — Creating Multiple Tasks and Priorities in CMSIS RTOS
- **Part 4** — Using Semaphores in CMSIS RTOS
- **Part 5** — Using Mutexes in CMSIS RTOS
- **Part 6** — Task Synchronization using Event Flags
- **Part 7** — Software Timers in FreeRTOS
- **Part 8** — FreeRTOS Stack Management

Table Of Contents

Why Global Variables Fail in FreeRTOS

Before we jump into queues, we need to understand why the simpler approach (using global variables) is a bad idea in FreeRTOS. Let's look at what actually goes wrong.

What Is a Race Condition?

A race condition happens when two or more tasks try to access the same data at the same time, and the final result depends on which task runs first. In a FreeRTOS system, the scheduler can switch between tasks at almost any point. This means you can never be sure which task is running at any given moment.

Say we have a structure like this:

```
1 typedef struct {
```

[HOME](#)[STM32](#) ▾[ESP32](#) ▾[Arduino](#) ▾[TIVA C](#)[AVR](#)[About](#)[Search](#) [English](#) ▾

```
5  
6 Message_t sharedData;
```

Task 1 starts writing to it:

```
1 sharedData.eventID = 1;  
2 sharedData.timestamp = HAL_GetTick();
```

But right in the middle of that write (after setting `eventID` but before setting `timestamp`) the scheduler switches to Task 2. Task 2 then overwrites the same structure with its own data:

```
1 sharedData.eventID = 2;  
2 sharedData.timestamp = HAL_GetTick();
```

Now when the consumer task reads `sharedData` , it gets a mix of values — `eventID` from Task 2, but maybe `timestamp` still half-written from Task 1. The data is corrupted, and you will likely never see a clear error. The system just behaves incorrectly.

The GIF below shows how this race condition plays out step by step:

Why volatile Does Not Fix the Race Condition

The problem with global variables is that FreeRTOS tasks run concurrently. There is no built-in protection when multiple tasks access a global variable at the same time. Any task can read or write it at any moment.

Here is what that looks like in practice:

```

3     for (;;) {
4         sharedData.eventID = 1;
5         sharedData.timestamp = HAL_GetTick();
6         osDelay(100);
7     }
8 }
9
10 // Task 2
11 void Task2(void *argument) {
12     for (;;) {
13         sharedData.eventID = 2;
14         sharedData.timestamp = HAL_GetTick() / 1000;
15         osDelay(1000);
16     }
17 }
18
19 // Consumer Task
20 void ConsumerTask(void *argument) {
21     for (;;) {
22         printf("ID: %d, Time: %lu\n",
23             sharedData.eventID,
24             sharedData.timestamp);
25         osDelay(500);
26     }
27 }

```

This looks harmless. But consider what happens when all three tasks are running:

- Task 1 and Task 2 are both writing to `sharedData`
- The consumer task is reading from it at the same time
- The scheduler decides when each task runs

The result is that the consumer task will sometimes read data that belongs to Task 1, sometimes Task 2, and sometimes a corrupted mix of both. The behavior is unpredictable and changes every time you run the program.

Making the variable `volatile` does **not** fix this. `volatile` only tells the compiler not to cache the variable — it does nothing to prevent two tasks from accessing it at the same time.

The only real solution is to use a communication mechanism that is designed for multi-task environments. In FreeRTOS, that mechanism is the **queue**. A queue ensures that each message is written and read atomically, so there is no risk of one task corrupting another task's data.

How a FreeRTOS Queue Works

Now that we understand the problem with global variables, let's look at the solution. FreeRTOS queues are built specifically for passing data between tasks safely. They handle all the timing and access issues internally, so we do not have to worry about race conditions.

Queue as a Thread-Safe Message Buffer

Think of a queue as a small buffer that sits between your tasks. One task writes a message into it, and

In our example, we will configure the queue to hold **10 elements**. You can imagine it as 10 slots, where each slot stores one complete message.

The GIF below shows how Task 1 and Task 2 push messages into the queue, and the receiver task reads from the other end:

Each message in our case is a structure, not just a simple integer. The queue stores one full copy of the structure per slot. So when Task 1 pushes its struct into the queue, it gets its own slot. When Task 2 pushes its struct, it gets the next slot. Neither message overwrites the other.

This is the key difference from a global variable. With a global variable, both tasks share one memory location. With a queue, every message gets its own dedicated space.

FIFO Order — First In, First Out

FreeRTOS queues follow the **FIFO** rule — First In, First Out. This means the first message that enters the queue is also the first one to come out.

So if Task 1 sends a message first, and Task 2 sends a message after that, the receiver task will always read Task 1's message first, then Task 2's message. The order is guaranteed.

Here is a simple way to think about it:

The receiver always reads from the front. New messages always go to the back. The GIF below shows the FIFO order in action, with Task 1's struct being read before Task 2's:

CMSIS-OS Queue — Message Priority & Timeout

So far we have looked at how queues deliver messages in FIFO order. But CMSIS-OS queues have one more feature worth understanding before we write any code — message priority. This affects the order in which messages are read, and the timeout option controls what happens when the queue has no room left.

Message Priority in `osMessageQueuePut()`

By default, messages follow strict FIFO order — first in, first out. But CMSIS-OS allows you to assign a priority value to each message when you call `osMessageQueuePut()`. A higher priority value means that message will jump ahead of lower-priority messages already waiting in the queue.

So if three messages are sitting in the queue and a new HIGH priority message arrives, it gets placed in front of the others — not at the back.

The GIF below shows this in action:

In our example, we will not be using priority-based messaging. We will simply pass `0` as the priority value, which means all messages are treated equally and delivered in normal FIFO order.

```
1 osMessageQueuePut(MessageQueueHandle, &message, 0, osWaitForever);
```

The `0` here is the priority parameter. All messages with the same priority follow FIFO order among themselves. So if both tasks send with priority `0`, the queue behaves exactly like a standard FIFO queue.

Timeout Options — 0, N ms, osWaitForever

The last parameter in `osMessageQueuePut()` is the timeout. This tells the function what to do if the queue is already full when you try to send a message.

You have three choices:

- `0` : Do not wait at all. If the queue is full, the function returns immediately with an error code. Use this when your task cannot afford to be blocked.
- **N (milliseconds)** : Wait up to N milliseconds for a free slot. If space opens up in time, the message is sent. If not, the function returns with an error.
- **osWaitForever** : Wait indefinitely. The task is blocked until a slot becomes free. The scheduler will run other tasks in the meantime, so no CPU time is wasted.

We will use `0` in this project, since there is enough space in the queue. Also the receiver task will run more frequently than the sender tasks, hence it will empty the queue more often.

```
1 osMessageQueuePut(MessageQueueHandle, &message, 0, 0);
```

The same timeout logic applies on the receiving side with `osMessageQueueGet()`. If the queue is empty and the timeout is `osWaitForever`, the receiver will simply wait until a message arrives. This is actually the cleanest way to write a receiver task — it blocks when there is nothing to read and wakes up the moment new data is available.

```
1 osMessageQueueGet(MessageQueueHandle, &message, NULL, osWaitForever);
```

With these two calls in place, the sender blocks when the queue is full and the receiver blocks when the queue is empty. Neither task wastes CPU cycles polling in a loop.

STM32 FreeRTOS Queue — CubeMX Configuration

Before we write any code, we need to set up the project in CubeMX. We will configure three tasks, create the message queue, and enable LPUART1 so we can route `printf` output to the serial console.

Create Three Tasks in CubeMX

Go to Middleware → FreeRTOS and open the Tasks and Queues tab. Here we will create three tasks for this example. Task1 and Task2 will update the data in the Queue, whereas the RxTask will read this data and print it on the serial console.

The images below shows the 3 Tasks in FreeRTOS:

Here is a quick breakdown of what each task does:

- **Task 1** will send data to the queue when a button is pressed. Set its priority to **Normal** and its stack size to **128 words**. This task does very little, it just loads data into a structure and pushes it to the queue, so 128 words is enough.
- **Task 2** also sends data to the queue, but it does so automatically every second. Give it the same **Normal** priority and the same stack size of **128 words**.
- **Task 3** is the receiver. It reads messages from the queue and prints them via UART using `printf`. Because `printf` is a heavier operation, we need a larger stack — set it to **512 words** (around 2 KB). We also want this task to have a higher priority than Task 1 and Task 2, so set it to **High**.

Why RxTask Gets Higher Priority and Larger Stack

The reason for the higher priority is important. If Task 1 or Task 2 preempts the receiver while it is

completion before the sender tasks get CPU time. Normally we would protect UART with a mutex or semaphore, but since we have not covered those yet, priority is the simplest solution for now.

After increasing the stack size for Task 3, you will likely see a heap error in CubeMX. Go to **Parameter Settings** and increase the heap size to around **5 KB**. The error should disappear.

Here is a summary of the task configuration:

Task	Priority	Stack Size	Role
Task 1	Normal	128 words	Sends struct on button press
Task 2	Normal	128 words	Sends struct every 1 second
Task 3	High	512 words	Reads queue, prints via UART

Create the MessageQueue in CubeMX

Still inside the Tasks and Queues tab, scroll down to the queue section and add a new queue. Name it MessageQueue .

Set the **queue size** to 10 . This means the queue can hold up to 10 messages at a time. Since two tasks are writing to it, 10 slots gives us enough headroom before the queue fills up.

For the **item size**, CubeMX will ask for a data type. Set it to `uint32_t` for now, we will change this manually in the code once we define our message structure. The actual structure we will use contains two fields:

- `eventID` — a `uint8_t` that identifies which task sent the message
- `timestamp` — a `uint32_t` that stores the tick count at the time of sending

Set the **allocation** to **Dvnamic** so the memory is taken automatically from the FreeRTOS heap.

Configure LPUART1 for printf Output

We will use `printf` to print the received data to a serial console. On the STM32L496 Nucleo board, the Virtual COM port routes UART data through the ST-LINK USB connection. Looking at the board schematic, ST-LINK RX is connected to **LPUART1 TX** and ST-LINK TX is connected to **LPUART1 RX**. These map to pins **PG7** and **PG8**.

Go to Connectivity and enable LPUART1 in Asynchronous mode. By default, CubeMX assigns pins PC0 and PC1, so change these to **PG7 (TX)** and **PG8 (RX)**.

Use the following settings:

Parameter	Value
Baud Rate	115200
Word Length	8 bits
Parity	None
Stop Bits	1

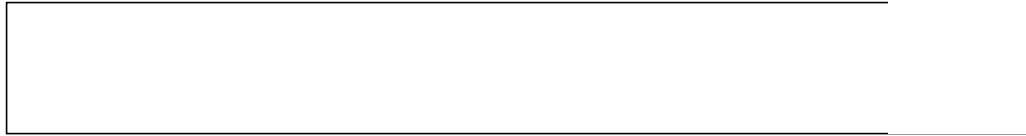
This is the standard UART configuration and matches what we will set in the serial console later.

Configure the Button on PA3

We also need a button for Task 1. I have connected the button between pin PA3 and Ground. The pin will be high by default, and when the button is pressed, it will be pulled low to the ground.

Configure pin PA3 as GPIO Input. Then open the GPIO settings for that pin and set it to Pull-Up mode.

The pin will stay HIGH by default, and when the button is pressed it will be pulled LOW to ground — that is the signal we will check inside Task 1.



STM32 FreeRTOS Queue Code — Sender & Receiver Tasks

With the project generated, we can now write the actual task code. We will define the message structure, wire up `printf` to UART, write the two sender tasks, and then write the receiver.

Define the Message Structure (`messageQueue_t`)

The first thing we need to do is define the structure that both tasks will use to send data. Add this near the top of the file, before the task function definitions:

```
1 typedef struct {
2     uint8_t event_id;
3     uint32_t timestamp;
4 } messageQueue_t;
```

`event_id` tells the receiver which task sent the message — we will use `1` for Task 1 and `2` for Task 2. `timestamp` stores the tick count at the moment the message was created, using `HAL_GetTick()`.

Now we need to update the queue to use this structure instead of `uint32_t`. Find the line where CubeMX created the queue — it will look something like this:



```
1 MessageQueueHandle = osMessageQueueNew(10, sizeof(uint32_t), &MessageQueue_attribute);
```

Change `sizeof(uint32_t)` to `sizeof(messageQueue_t)` :

```
1 MessageQueueHandle = osMessageQueueNew(10, sizeof(messageQueue_t), &MessageQueue_attribute);
```

The queue still holds 10 elements, but now each element is one full `messageQueue_t` structure instead of a single 32-bit integer.

Route printf Through LPUART1 (__io_putchar)

To use `printf` for serial output, we need to redirect it through UART. Add this function to `main.c` :

```
1 int __io_putchar(int ch)
2 {
3     HAL_UART_Transmit(&hlpuart1, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
4     return ch;
5 }
```

Every time `printf` outputs a character, it calls `__io_putchar()` internally. We override it here to transmit that character over LPUART1.

Also make sure to add this include at the top of `main.c`, otherwise the compiler will complain about `printf` being undefined:

```
1 #include <stdio.h>
```

Task1 — Send Data on Button Press

Task 1 monitors the button on pin PA3. When pressed, it loads data into a `messageQueue_t` structure and pushes it into the queue.

```
1 void StartTask1(void *argument)
2 {
3     messageQueue_t msg;
4
5     for (;;)
6     {
7         if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3) == GPIO_PIN_RESET)
8         {
9             msg.event_id = 0x01;
10            msg.timestamp = HAL_GetTick();
11
12            osMessageQueuePut(MessageQueueHandle, &msg, 0, 0);
13
14            osDelay(200);
15        }
16        osDelay(20);
17    }
```

A few things to note here. We check for `GPIO_PIN_RESET` because the pin is pulled HIGH by default and goes LOW when the button is pressed. We set `event_id` to `1` so the receiver knows this message came from Task 1. The timestamp is in milliseconds, straight from `HAL_GetTick()`.

We pass `0` as the message priority as this is a standard FIFO message with no special priority. We use `0` as the timeout, so if the queue is full the task will simply skip that particular message and the loop runs again.

The `osDelay(200)` at the end prevents the button debouncing.



Task2 — Send Data Automatically Every Second

The Task 2 does not wait for a button press. It automatically sends a message to the queue every second. We use a different `event_id` and convert the timestamp to seconds so the output is easy to distinguish from Task 1.

```
1 void StartTask2(void *argument)
2 {
3     messageQueue_t msg;
4
5     for (;;)
6     {
7         msg.event_id = 2;
8         msg.timestamp = HAL_GetTick() / 1000;
9
10        osMessageQueuePut(MessageQueueHandle, &msg, 0, 0);
11
12        osDelay(1000);
13    }
14 }
```

This task runs independently of Task 1. So even when the button is not pressed, Task 2 keeps sending data every second. When the button is pressed, both tasks are sending at the same time — and the queue handles both safely without any race conditions.

RxTask — Read Queue and Print via UART

The receiver task reads messages from the queue one at a time and prints them to the serial console. It uses `osWaitForever`, so it blocks when the queue is empty and wakes up the moment a new message

```

1 void StartRxTask(void *argument)
2 {
3     messageQueue_t msg;
4     for(;;)
5     {
6         if (osMessageQueueGet(messageQueueHandle, &msg, 0, osWaitForever) == osOK)
7         {
8             printf ("Event ID: %d, Timestamp: %lu\n",
9                     msg.event_id, msg.timestamp);
10        }
11        osDelay(1);
12    }
13 }

```

We check `status == osOK` before printing. This ensures we only print when the read was successful. With `osWaitForever`, the function will always return `osOK` once it gets a message, but it is good practice to check anyway.

Notice there is no `osDelay()` here. The task does not need one because `osMessageQueueGet()` with `osWaitForever` already blocks the task when the queue is empty. The scheduler runs other tasks during that time, so no CPU cycles are wasted.

Full Combined Code

Here is the complete `main.c` with everything put together:

```

1 #include "main.h"
2 #include "cmsis_os.h"
3 #include <stdio.h>
4
5 /* Queue handle - generated by CubeMX */
6 extern osMessageQueueId_t MessageQueueHandle;
7 extern UART_HandleTypeDef hlpuart1;
8
9 /* Message structure */
10 typedef struct {
11     uint8_t event_id;
12     uint32_t timestamp;
13 } messageQueue_t;
14
15 /* Redirect printf to UART */
16 int __io_putchar(int ch)
17 {
18     HAL_UART_Transmit(&hlpuart1, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
19     return ch;
20 }
21
22 /* Task 1 - Send on button press */
23 void StartTask1(void *argument)
24 {
25     messageQueue_t msg;
26     for(;;)
27     {
28         if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_3) == 0)
29         {
30             msg.event_id = 0x01;
31             msg.timestamp = HAL_GetTick();
32             osMessageQueuePut(MessageQueueHandle, &msg, 0, 0);
33             osDelay(200); // debounce

```

```

37 }
38
39 /* Task 2 - Send every second */
40 void StartTask2(void *argument)
41 {
42     messageQueue_t msg;
43     for(;;)
44     {
45         msg.event_id = 0x02;
46         msg.timestamp = HAL_GetTick()/1000; // in seconds
47         osMessageQueuePut(messageQueueHandle, &msg, 0, 0);
48         osDelay(1000);
49     }
50 }
51
52 /* Task 3 - Receive and print */
53 void StartRxTask(void *argument)
54 {
55     messageQueue_t msg;
56     for(;;)
57     {
58         if (osMessageQueueGet(messageQueueHandle, &msg, 0, osWaitForever) == osOK)
59         {
60             printf ("Event ID: %d, Timestamp: %lu\n", msg.event_id, msg.timestamp);
61         }
62         osDelay(1);
63     }
64 }

```

This is the core of the application. The rest of `main.c` — the HAL init, clock config, and CubeMX-generated queue and task creation code — stays exactly as generated. We only add the structure definition, the `__io_putchar` function, and the three task bodies shown above.



Output — Serial Console with Task1 and Task2 Messages

With the code written and the project built successfully, it is time to flash it to the board and see it in action. Open your serial console. Make sure the settings match what we configured in CubeMX.

Once connected, you should immediately see output from Task 2 printing every second. And when the button is pressed, the data from Task1 will print alongside.

The image below shows the serial console output with both tasks sending data simultaneously:

Notice that the messages never overlap or corrupt each other. Task 1 and Task 2 are both writing to the queue at the same time, and the receiver reads them cleanly in order. This is exactly the problem that global variables cannot solve — but a queue handles it without any issues.

CMSIS-OS V2 Queue API Reference

Now that our queue is working, let us look at some additional CMSIS-OS functions that come in handy when working with queues. We will not use all of these in this project, but knowing they exist will save you a lot of time as your projects get more complex.

Here is a quick overview of the available functions:

Function	What it returns
osMessageQueueGetCapacity()	Total number of slots the queue can hold
osMessageQueueGetMsgSize()	Size of each message in bytes inside the queue
osMessageQueueGetCount()	Number of messages currently waiting in the queue
osMessageQueueGetSpace()	Number of free slots currently available in the queue
osMessageQueueReset()	Clears all messages from the queue

STM32 FreeRTOS Queue Tutorial — Video Walkthrough



This video covers the complete FreeRTOS queue workflow on STM32: race condition explanation with global variables, queue FIFO and message priority concepts, CubeMX configuration for three tasks and a MessageQueue, LPUART1 printf routing, Task1 button-triggered sender, Task2 periodic sender, and RxTask receiver with osWaitForever — all verified on the STM32 Nucleo-L496ZG-P serial console.

STM32 FreeRTOS Queue — Frequently Asked Questions

+ Can I send different types of data through the same queue?

+ Is it safe to use the same queue from both a task and an interrupt?

+ Does increasing the queue size affect real-time performance?

+ Can two tasks read from the same queue simultaneously?

Conclusion

The FreeRTOS queue is the cleanest inter-task communication primitive for the most common embedded pattern: one or more tasks producing data, one task consuming it. Every message gets its own atomic slot, copies are made on both send and receive, and the scheduler handles blocking and unblocking automatically — no critical sections, no `volatile` hacks, no global variable corruption.

In this tutorial you built the complete pattern from scratch: defined a typed message structure, sized the queue for 10 messages, configured a higher-priority receiver to ensure UART output integrity, wrote a button-triggered sender and a periodic sender, and implemented the receiver with `osWaitForever` — so it blocks cleanly when the queue is empty and wakes instantly when a message arrives.

The natural limitation of this design is the single receiver. What happens when two tasks both need to access the same UART handle at the same time? `printf` inside a FreeRTOS task is not thread-safe by default — the receiver's higher priority handles it here, but that is a workaround, not a solution. The correct tool is a **mutex**: a lock that ensures only one task can hold the UART handle at any moment. That is exactly what [Part 4 — Semaphores](#) and [Part 5 — Mutexes](#) cover next. Browse the full **STM32 FreeRTOS series** for all parts.

Download STM32 FreeRTOS Queue Project Files

Complete STM32CubeIDE project for the STM32 Nucleo-L496ZG-P with CMSIS-OS V2 queue implementation: `messageQueue_t` struct definition, 10-slot `MessageQueue`, Task1 button-triggered sender (PA3). Task2 periodic 1-second sender. RxTask high-



printf output at 115200 baud. Free to download —
support the work if it helped you.

[Open source](#)[CMSIS-RTOS V2 + FreeRTOS](#)[Queue · 2 Senders + Receiver](#)[CubeMX + HAL source](#)[View on GitHub](#)[Support my work](#)

Browse More STM32 FreeRTOS Tutorials

**STM32 FreeRTOS Tutorial:
CMSIS-RTOS V2 Setup,
Tasks & LED Blink in
CubeMX**

**STM32 FreeRTOS Multiple
Tasks, Priorities &
Preemption — CMSIS-
RTOS V2 Guide**

**STM32 FreeRTOS
Semaphores: How to Use
Binary and Counting
Semaphores**

STM32 FreeRTOS Mutex:
Priority Inheritance &
Recursive Mutex

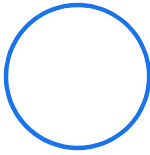
STM32 FreeRTOS Event
Flags: osFlagsWaitAll,
WaitAny & ISR Callbacks

STM32 FreeRTOS Software
Timers: Periodic & One-
Shot with CMSIS-OS V2

1 2 Next »



ABOUT THE AUTHOR



Arun Rawat

EMBEDDED SYSTEMS ENGINEER · FOUNDER,
CONTROLLERSTECH

Arun is an embedded systems engineer with 10+ years of experience in STM32, ESP32, and AVR microcontrollers. He created ControllersTech to share practical tutorials on embedded software, HAL drivers, RTOS, and hardware design — grounded in real industrial automation experience.

[About](#)[YouTube](#)[GitHub](#)[Facebook](#)[Instagram](#)[Shop](#)

✉ Subscribe ▼

Login

{}
[+]



0 COMMENTS

Subscribe to Our Newsletter

Email

☐ I accept the privacy policy

Subscribe



© 2026 ControllersTech® · All Rights Reserved · Built with ❤️ for Embedded Engineers

[About Me](#)

[Privacy Policy](#)

[SHOP](#)

[Contact US](#)

[Do Not Sell or Share My Personal Information](#)